WEEK 9 - SALES AND CUSTOMER ANALYTICS SYSTEM

The Photo Frame Shopping System maintains customer, product, category, and order details. SQL queries are used to generate business analytics reports and identify sales performance, customer spending, best-selling products, and category-wise revenue.

GENERATES TOTAL SALES REPORTS :

1.TOTAL NUMBER OF ORDERS PLACED :

```
SQL> SELECT COUNT(*)
  2  FROM ORDER_ENTITY;

  COUNT(*)
----------
        10
```

```
SQL> SELECT COUNT(*) AS TOTAL_ORDERS
  2  FROM ORDER_ENTITY;

TOTAL_ORDERS
------------
          10
```

2.TOTAL REVENUE GENERATED FROM SALES

```
SQL> SELECT SUM(TOTAL_AMOUNT)
  2  FROM ORDER_ENTITY;

SUM(TOTAL_AMOUNT)
-----------------
            19300
```

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE FROM ORDER_ENTITY;

TOTAL_REVENUE
-------------
        19300
```

3.AVERAGE ORDER VALUES

```
SQL> SELECT AVG(TOTAL_AMOUNT) FROM ORDER_ENTITY;

AVG(TOTAL_AMOUNT)
-----------------
             1930
```

4.AVERAGE ORDERS VALUES OF EACH ORDER AT A TIME

```
SQL> SELECT AVG(TOTAL_AMOUNT) AS AVG_ORDER FROM ORDER_ENTITY
  2  GROUP BY ORDER_ID;

 AVG_ORDER
----------
      1500
      2200
      1800
      2500
      1200
      1700
      1900
      2300
      1400
      2800

10 rows selected.
```

4.HIGHEST AND LOWEST ORDER AMOUNT

I.HIGHEST AMOUNT :

```
SQL> SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_AMOUNT  FROM ORDER_ENTITY;

HIGHEST_AMOUNT
--------------
          2800
```

II. LOWEST AMOUNT:

```
SQL> SELECT MIN(TOTAL_AMOUNT ) AS LOWEST_AMOUNT FROM ORDER_ENTITY

LOWEST_AMOUNT
-------------
         1200
```

5.TOTAL SALES FOR A SPECIFIC PERIOD :

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES FROM ORDER_ENTITY
  2  WHERE ORDER_DATE  BETWEEN
  3  TO_DATE('20-09-2026','DD-MM-YYYY')
  4  AND TO_DATE('25-09-2026','DD-MM-YYYY');

TOTAL_SALES
-----------
      10900
```

3. Customer Purchase Analysis

1. Total number of orders placed by each customer.

```
SQL>  SELECT C.CUSTOMER_ID,C. CUSTOMER_NAME ,
  2  COUNT(O.ORDER_ID) AS TOTAL_ORDER
  3  FROM CUSTOMER C
  4  JOIN ORDER_ENTITY O
  5  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  6  GROUP BY C.CUSTOMER_ID ,C.CUSTOMER_NAME;

CUSTOMER_ID CUSTOMER_N TOTAL_ORDER
----------- ---------- -----------
        204 DIVYA                1
        206 MEENA                1
        207 VIGNESH              1
        209 RAHUL                1
        202 PRIYA                1
        203 KARTHIK              1
        205 SURESH               1
        201 ARUN                 1
        208 ANU                  1
        210 NISHA                1

10 rows selected.
```

2. Total amount spent by each customer.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENT
  4  FROM CUSTOMER C
  5  JOIN ORDER_ENTITY O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME;

CUSTOMER_ID CUSTOMER_N TOTAL_SPENT
----------- ---------- -----------
        204 DIVYA             2500
        206 MEENA             1700
        207 VIGNESH           1900
        209 RAHUL             1400
        202 PRIYA             2200
        203 KARTHIK           1800
        205 SURESH            1200
        201 ARUN              1500
        208 ANU               2300
        210 NISHA             2800

10 rows selected.
```

3. Average spending per customer.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         AVG(O.TOTAL_AMOUNT) AS AVG_SPENDING
  4  FROM CUSTOMER C
  5  JOIN ORDER_ENTITY O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME;

CUSTOMER_ID CUSTOMER_N AVG_SPENDING
----------- ---------- ------------
        204 DIVYA              2500
        206 MEENA              1700
        207 VIGNESH            1900
        209 RAHUL              1400
        202 PRIYA              2200
        203 KARTHIK            1800
        205 SURESH             1200
        201 ARUN               1500
        208 ANU                2300
        210 NISHA              2800

10 rows selected.
```

4. Customers with maximum purchase amount.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME,
  2         SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENT
  3  FROM CUSTOMER C
  4  JOIN ORDER_ENTITY O
  5  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  6  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  7  HAVING SUM(O.TOTAL_AMOUNT) = (
  8      SELECT MAX(TOTAL_SPENT)
  9      FROM (
 10          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENT
 11          FROM ORDER_ENTITY
 12          GROUP BY CUSTOMER_ID
 13      )
 14  );

CUSTOMER_ID CUSTOMER_N TOTAL_SPENT
----------- ---------- -----------
        210 NISHA             2800
```

5. Customers with fewer purchases.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME,
  2  COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS
  3  FROM CUSTOMER C
  4  JOIN ORDER_ENTITY O
  5  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  6  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME

CUSTOMER_ID CUSTOMER_N NUMBER_OF_ORDERS
----------- ---------- ---------------
        204 DIVYA                    1
        206 MEENA                    1
        207 VIGNESH                  1
        209 RAHUL                    1
        202 PRIYA                    1
        203 KARTHIK                  1
        205 SURESH                   1
        201 ARUN                     1
        208 ANU                      1
        210 NISHA                    1

10 rows selected.
```

4. Find Top Customers Based on Purchase Amount

1. Top 5 customers based on total spending.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME,
  2  SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING
  3  FROM CUSTOMER C
  4  JOIN ORDER_ENTITY O
  5  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  6  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  7  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID CUSTOMER_N TOTAL_SPENDING
----------- ---------- --------------
        210 NISHA                2800
        204 DIVYA                2500
        208 ANU                  2300
        202 PRIYA                2200
        207 VIGNESH              1900
```

2. Customers with maximum number of orders.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME,
  2  COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  3  FROM CUSTOMER C
  4  JOIN ORDER_ENTITY O
  5  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  6  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  7  ORDER BY TOTAL_ORDERS DESC;

CUSTOMER_ID CUSTOMER_N TOTAL_ORDERS
----------- ---------- ------------
        204 DIVYA                 1
        206 MEENA                 1
        207 VIGNESH               1
        209 RAHUL                 1
        210 NISHA                 1
        203 KARTHIK               1
        205 SURESH                1
        201 ARUN                  1
        208 ANU                   1
        202 PRIYA                 1

10 rows selected.
```

3. Frequent customers.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME,
  2  COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  3  FROM CUSTOMER C
  4  JOIN ORDER_ENTITY O
  5  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  6  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  7  HAVING COUNT(O.ORDER_ID) > 1;

no rows selected
```

4. High-value customers.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME,
  2  COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS,
  3  SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING
  4  FROM CUSTOMER C
  5  JOIN ORDER_ENTITY O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME;

CUSTOMER_ID CUSTOMER_N NUMBER_OF_ORDERS TOTAL_SPENDING
----------- ---------- ---------------- --------------
        204 DIVYA                     1           2500
        206 MEENA                     1           1700
        207 VIGNESH                   1           1900
        209 RAHUL                     1           1400
        202 PRIYA                     1           2200
        203 KARTHIK                   1           1800
        205 SURESH                    1           1200
        201 ARUN                      1           1500
        208 ANU                       1           2300
        210 NISHA                     1           2800

10 rows selected.
```

5. Identify Best-Selling Products

1. Products with maximum sales quantity.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME,
  2  SUM(OD.QUANTITY) AS QUANTITY_SOLD,
  3  SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE
  4  FROM PRODUCT P
  5  JOIN ORDER_DETAILS_ENTITY OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME;

PRODUCT_ID PRODUCT_NAME                                       QUANTITY_SOLD
---------- -------------------------------------------------- -------------
TOTAL_REVENUE
-------------
       104 Decorative Wall Frame                                         6
        13000

       101 Classic Wooden Frame                                          4
         4800

       102 Elegant Acrylic Frame                                         5
         9100


PRODUCT_ID PRODUCT_NAME                                       QUANTITY_SOLD
---------- -------------------------------------------------- -------------
TOTAL_REVENUE
-------------
       103 Family Collage Frame                                          4
         7200

       105 Table Photo Frame                                             6
         7200
```

3. Least-selling products.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME,
  2  SUM(OD.QUANTITY) AS QUANTITY_SOLD
  3  FROM PRODUCT P
  4  JOIN ORDER_DETAILS_ENTITY OD
  5  ON P.PRODUCT_ID = OD.PRODUCT_ID
  6  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  7  ORDER BY QUANTITY_SOLD ASC;

PRODUCT_ID PRODUCT_NAME                                       QUANTITY_SOLD
---------- -------------------------------------------------- -------------
       103 Family Collage Frame                                          4
       101 Classic Wooden Frame                                          4
       102 Elegant Acrylic Frame                                         5
       104 Decorative Wall Frame                                         6
       105 Table Photo Frame                                             6
```

4. Products requiring promotion.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME,
  2  SUM(OD.QUANTITY) AS QUANTITY_SOLD
  3  FROM PRODUCT P
  4  JOIN ORDER_DETAILS_ENTITY OD
  5  ON P.PRODUCT_ID = OD.PRODUCT_ID
  6  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  7  HAVING SUM(OD.QUANTITY) < 3;

no rows selected
```

6. Perform Category-wise Sales Analysis

1. Total sales generated by each category.

```
SQL> SELECT C.CATEGORY_ID, C.CATEGORY_NAME,
  2  SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_SALES
  3  FROM CATEGORY C
  4  JOIN PRODUCT P
  5  ON C.CATEGORY_ID = P.CATEGORY_ID
  6  JOIN ORDER_DETAILS_ENTITY OD
  7  ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME;

CATEGORY_ID CATEGORY_NAME                                      TOTAL_SALES
----------- -------------------------------------------------- -----------
          1 Wooden Photo Frames                                       4800
          2 Acrylic Photo Frames                                      9100
          3 Collage Photo Frames                                      7200
          4 Wall Mounted Frames                                      13000
          5 Tabletop Photo Frames                                     7200
```

2. Number of products sold in each category.

```
SQL> SELECT C.CATEGORY_ID, C.CATEGORY_NAME,
  2  SUM(OD.QUANTITY) AS PRODUCTS_SOLD
  3  FROM CATEGORY C
  4  JOIN PRODUCT P
  5  ON C.CATEGORY_ID = P.CATEGORY_ID
  6  JOIN ORDER_DETAILS_ENTITY OD
  7  ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME;

CATEGORY_ID CATEGORY_NAME                                      PRODUCTS_SOLD
----------- -------------------------------------------------- -------------
          1 Wooden Photo Frames                                            4
          2 Acrylic Photo Frames                                           5
          3 Collage Photo Frames                                           4
          4 Wall Mounted Frames                                            6
          5 Tabletop Photo Frames                                          6
```

3. Highest revenue-generating category.

```
SQL> SELECT C.CATEGORY_ID, C.CATEGORY_NAME,
  2  SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE
  3  FROM CATEGORY C
  4  JOIN PRODUCT P
  5  ON C.CATEGORY_ID = P.CATEGORY_ID
  6  JOIN ORDER_DETAILS_ENTITY OD
  7  ON P.PRODUCT_ID = OD.PRODUCT_ID
  8  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
  9  ORDER BY TOTAL_REVENUE DESC;

CATEGORY_ID CATEGORY_NAME                                      TOTAL_REVENUE
----------- -------------------------------------------------- -------------
          4 Wall Mounted Frames                                        13000
          2 Acrylic Photo Frames                                        9100
          5 Tabletop Photo Frames                                       7200
          3 Collage Photo Frames                                        7200
          1 Wooden Photo Frames                                         4800
```

7. Generate Business Analytics Reports

Report 1: Sales Performance Report

- Total sales

- Number of orders
- Average order value Highest order value

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES,
  2         COUNT(ORDER_ID) AS NUMBER_OF_ORDERS,
  3         AVG(TOTAL_AMOUNT) AS AVG_ORDER_VALUE,
  4         MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_VALUE
  5  FROM ORDER_ENTITY;

TOTAL_SALES NUMBER_OF_ORDERS AVG_ORDER_VALUE HIGHEST_ORDER_VALUE
----------- ---------------- --------------- -------------------
      19300               10            1930                2800
```

Report 2: Customer Analytics Report •

- Customer name
- Number of orders
- Total spending
- Purchase frequency
-

```
SQL> SELECT C.CUSTOMER_NAME,
  2         COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING,
  4         COUNT(O.ORDER_ID) AS PURCHASE_FREQUENCY
  5  FROM CUSTOMER C
  6  JOIN ORDER_ENTITY O
  7  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  8  GROUP BY C.CUSTOMER_NAME;

CUSTOMER_N NUMBER_OF_ORDERS TOTAL_SPENDING PURCHASE_FREQUENCY
---------- ---------------- -------------- ------------------
ANU                       1           2300                  1
RAHUL                     1           1400                  1
NISHA                     1           2800                  1
KARTHIK                   1           1800                  1
VIGNESH                   1           1900                  1
PRIYA                     1           2200                  1
MEENA                     1           1700                  1
ARUN                      1           1500                  1
DIVYA                     1           2500                  1
SURESH                    1           1200                  1

10 rows selected.
```

Report 3: Product Performance Report

- Product name
- Quantity sold
- Revenue generated

```
SQL> SELECT P.PRODUCT_NAME,
  2         SUM(OD.QUANTITY) AS QUANTITY_SOLD,
  3         SUM(OD.QUANTITY * OD.PRICE) AS REVENUE_GENERATED
  4  FROM PRODUCT P
  5  JOIN ORDER_DETAILS_ENTITY OD
  6  ON P.PRODUCT_ID = OD.PRODUCT_ID
  7  GROUP BY P.PRODUCT_NAME;

PRODUCT_NAME                                       QUANTITY_SOLD
-------------------------------------------------- -------------
REVENUE_GENERATED
-----------------
Decorative Wall Frame                                          6
            13000

Table Photo Frame                                              6
             7200

Classic Wooden Frame                                           4
             4800


PRODUCT_NAME                                       QUANTITY_SOLD
-------------------------------------------------- -------------
REVENUE_GENERATED
-----------------
Elegant Acrylic Frame                                          5
             9100

Family Collage Frame                                           4
             7200
```

Report 4: Category Analysis Report

- Category name
- Total products sold
- Total revenue

```
SQL> SELECT C.CATEGORY_NAME,
  2         SUM(OD.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
  3         SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE
  4  FROM CATEGORY C
  5  JOIN PRODUCT P
  6  ON C.CATEGORY_ID = P.CATEGORY_ID
  7  JOIN ORDER_DETAILS_ENTITY OD
  8  ON P.PRODUCT_ID = OD.PRODUCT_ID
  9  GROUP BY C.CATEGORY_NAME;

CATEGORY_NAME                                      TOTAL_PRODUCTS_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Collage Photo Frames                                               4
         7200

Wooden Photo Frames                                                4
         4800

Tabletop Photo Frames                                              6
         7200


CATEGORY_NAME                                      TOTAL_PRODUCTS_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Acrylic Photo Frames                                               5
         9100

Wall Mounted Frames                                                6
        13000
```